Despliegue real en AWS Academy — Pedidos360 Evaluación 2

> Este documento describe **el despliegue que quedó efectivamente funcionando** en la cuenta AWS Academy (alumno) `654654330164`, región `us-east-1`, usando **solo recursos de capa gratuita**. Todos los valores aquí escritos fueron verificados con comandos reales el **2026-11-02**. Si algo no coincide con `docs/GUIA_EVALUACION_2.md`, **manda este documento**.

---

1. Resumen de recursos creados

Recurso	Identificador / valor	Costo
Instancia EC2	`i-03a70f0a716b69333` — `t3.micro`, Amazon Linux 2023, 20 GB gp3	Free tier (750 h/mes)
Elastic IP	`32.200.126.178` (`eipalloc-04b1dfc182682f987`)	Gratis mientras esté asociada a la instancia
DNS público	`ec2-32-200-126-178.compute-1.amazonaws.com`	Gratis
Security Group	`sg-0e1a6c1707b3096a4`	Gratis
Key pair	`vockey` (precreado por el lab, archivo `labsuser.pem`)	Gratis
CloudFormation	stack `pedidos360-eval2`	Gratis
Cognito User Pool	`us-east-1_9b2C1fxtb`	Free tier (50 000 MAU)
Cognito App Client	`261rvdvemjgib6n86vrb05tl9k` (`pedidos360-spa-v2`)	Gratis
Cognito Domain	`https://pedidos360-b2c1fxtb.auth.us-east-1.amazoncognito.com`	Gratis
API Gateway HTTP API	`bf0plxi897`, stage `prod`	Free tier (1 M llamadas/mes, 12 meses)
Repositorio GitHub	`https://github.com/Kenpina0/pedidos360-eval2`	Gratis
GitHub Actions	pipeline CI/CD (3 jobs) + imagen en GHCR	Gratis en repo público

No se creó: ALB, NAT Gateway, RDS, ElastiCache, EKS, Route 53, ni instancias extra. Todo corre en **una sola EC2** con `docker compose` (backend + 2 nodos RabbitMQ + frontend + nginx TLS).

Puertos abiertos en el Security Group

Puerto	Servicio
22	SSH
80	frontend HTTP (interno / pruebas)
8080	backend Spring Boot (directo, sin gateway)
5173	frontend servido por nginx (HTTP)
8443	**frontend HTTPS** (nginx con certificado autofirmado) — el que se usa para el login
5672 / 5673	AMQP nodo 1 / nodo 2
15672 / 15673	Consola de administración RabbitMQ nodo 1 / nodo 2

---

2. Credenciales de AWS Academy (lo que hay que entender)

Tu cuenta es **AWS Academy Learner Lab**, no una cuenta AWS normal. Eso implica:

1. **No hay facturación**: el lab tiene un presupuesto fijo y las credenciales son temporales.

2. **No puedes crear roles IAM**. El lab ya trae `LabRole` y el perfil de instancia `LabInstanceProfile`. Por eso el stack de CloudFormation usa `IamInstanceProfile: LabInstanceProfile` en vez de crear uno.
3. **Las credenciales de la consola duran ~4 horas** y hay que refrescarlas con el botón *AWS Details*.
4. **La región es fija**: `us-east-1`.
5. **La key pair es fija**: `vockey`, y el archivo privado es `labsuser.pem`.

2.1 Por qué se usó CloudFormation y no la CLI directamente

El `aws_session_token` que llegó por chat estaba **truncado** (756 caracteres; al decodificar base64 quedaban 567 bytes y la firma se cortaba a la mitad), por lo que `aws sts get-caller-identity` devolvía `InvalidClientTokenId`. La solución fue:

1. Abrir la consola con la URL de federación del lab (`ssourl.txt`).
2. Crear la instancia con un **stack de CloudFormation** (`infra/academy/stack-ec2.yaml`) desde la consola.
3. Entrar por SSH con `labsuser.pem`.
4. **Leer credenciales válidas desde el metadata service (IMDSv2)** de la propia instancia, que hereda `LabRole`:

```
TOKEN=$(curl -s -X PUT "http://169.254.169.254/latest/api/token" \
  -H "X-aws-ec2-metadata-token-ttl-seconds: 21600")
curl -s -H "X-aws-ec2-metadata-token: $TOKEN" \
  http://169.254.169.254/latest/meta-data/iam/security-credentials/LabRole
```

Con esas credenciales (`ASIAZQ3DQKU2NOHOVYCA`, rol `LabRole`) se pudo usar la CLI para Cognito, API Gateway y las verificaciones. **Expiran a las 2026-11-02T07:23:28Z.**

> Si necesitas repetir el despliegue desde cero: refresca las credenciales en *AWS Details*, pégalas completas (las tres líneas: access key, secret key y **session token entero**), y ejecuta los scripts de `infra/academy/` en orden.

---

3. Cómo está armado el stack en la instancia

`infra/docker-compose.yml` levanta 5 contenedores:

Contenedor	Imagen	Puerto host	Rol
`rabbitmq-1`	`rabbitmq:3.13.7-management`	5672 / 15672	Nodo 1 del clúster (disco)
`rabbitmq-2`	`rabbitmq:3.13.7-management`	5673 / 15673	Nodo 2 del clúster (disco)
`pedidos360-backend`	build local / GHCR	8080	Spring Boot 3.3.5, Java 17
`pedidos360-frontend`	build local / GHCR	5173	React 18 + Vite servido por nginx
`pedidos360-frontend-tls`	`nginx:1.27-alpine`	**8443**	Termina TLS y hace proxy al frontend

3.1 Por qué existe `frontend-tls`

Cognito **rechaza callbacks con `http://` cuando el host no es `localhost`**:

```
InvalidParameterException: ... cannot use the HTTP protocol
```

Por eso el frontend se publica en `https://32.200.126.178:8443` con un certificado autofirmado (`subjectAltName = IP:32.200.126.178`). El navegador mostrará una advertencia de certificado: **es esperado, se acepta con "Avanzado → Continuar"**.

3.2 Límites de memoria (la instancia tiene solo 913 MB de RAM)

```
backend:
  environment:
    JAVA_TOOL_OPTIONS: "-Xmx256m -XX:MaxMetaspaceSize=160m"
  mem_limit: 460m
```

Además se agregó **2 GB de swap** en la instancia:

```
sudo dd if=/dev/zero of=/swapfile bs=1M count=2048
sudo chmod 600 /swapfile && sudo mkswap /swapfile && sudo swapon /swapfile
```

Sin el swap, los dos nodos de RabbitMQ + la JVM hacían que el OOM killer matara contenedores.

---

4. RabbitMQ — mapeo con la rúbrica

Verificado con `rabbitmqctl cluster_status` y `rabbitmqctl list_queues`:

- **Clúster de 2 nodos**: `rabbit@rabbitmq-1` + `rabbit@rabbitmq-2`, `cluster_name = pedidos360-cluster`, `disk_nodes` = ambos, `partitions: {}` (sin particiones de red).
- **3 colas de negocio**: `notificaciones.queue`, `tickets.queue`, `documentos.queue`.
- **3 colas de mensajes muertos**: `notificaciones.dlq`, `tickets.dlq`, `documentos.dlq`.
- **Exchanges**: `pedidos.direct` (direct), `pedidos.topic` (topic), `pedidos.dlx` (direct, DLX).
- **Política de alta disponibilidad**: `ha-all` con patrón `^(?!amq\.)` → `{"ha-mode":"all","ha-sync-mode":"automatic"}`.
- **Consumidores activos**: 1 en cada `.queue` (3 en total).

4.1 Detalle importante: `definitions.json`

El archivo `infra/rabbitmq/definitions.json` se importa al arrancar. Dos cosas que costaron sangre:

1. `"parameters"` **debe ser una lista** (`[]`), no un objeto (`{}`). Si es `{}`, RabbitMQ falla con `error:badarg ... erlang:length/1 args: [#{}]` y el nodo no arranca.
2. Al importar definiciones **se desactiva la creación del usuario por defecto**, así que el usuario `pedidos` debe venir en el JSON con su hash real:

```
"users": [{
  "name": "pedidos",
  "password_hash": "7CPBC6No0DVVdJw5LLUmYbq0A+5uH196TaG2HXQDkUPRYsXE",
  "hashing_algorithm": "rabbit_password_hashing_sha256",
  "tags": "administrator"
}]
```

El hash corresponde a la contraseña `pedidos123` y se calcula como `base64( salt[4 bytes] || sha256(salt || password) )`.

3. **El nodo 2 también necesita el volumen** de `definitions.json`, porque `rabbitmq.conf` lo referencia y falla con `management.load_definitions invalid, file does not exist`.

4.2 Demostración de DLQ

Un pedido con `cantidad = 150` supera el umbral de validación del consumidor de tickets, que hace `basicNack(requeue=false)`. El mensaje cae en `pedidos.dlx` y de ahí a `tickets.dlq`. Evidencia: `tickets.dlq` con **1 mensaje** y `tickets.queue` en 0 (ver §6).

---

5. Cognito + API Gateway

5.1 User Pool

- Pool: `us-east-1_9b2C1fxtb`
- Grupos: `ADMIN`, `COLLABORATOR`, `CUSTOMER`
- Usuarios (todos `CONFIRMED`):

Usuario	Contraseña	Grupo
`admin@pedidos360.cl`	`Admin1234`	ADMIN
`collab@pedidos360.cl`	`Collab1234`	COLLABORATOR
`cliente@pedidos360.cl`	`Cliente1234`	CUSTOMER

> Ojo: el usuario es `cliente@`, **no** `customer@`.

- App Client: `261rvdvemjgib6n86vrb05tl9k` (`pedidos360-spa-v2`), tipo SPA:
- Sin client secret
- `AllowedOAuthFlows: code` (Authorization Code + PKCE)
- `ExplicitAuthFlows: ALLOW_USER_PASSWORD_AUTH` , `ALLOW_REFRESH_TOKEN_AUTH`
- Callbacks: `https://32.200.126.178:8443` , `http://localhost:5173`
- Scopes: `openid email profile`

> Se tuvo que **crear un cliente nuevo** porque el cliente original rechazaba los callbacks con `redirect_mismatch` incluso después de guardarlos correctamente.

5.2 API Gateway

- HTTP API `bf0plxi897` , stage `prod`
- URL: `https://bf0plxi897.execute-api.us-east-1.amazonaws.com/prod`
- Autorizador JWT `yl7d6v` :
- Issuer: `https://cognito-idp.us-east-1.amazonaws.com/us-east-1_9b2C1fxtb`
- Audience: `261rvdvemjgib6n86vrb05tl9k`
- Rutas:

Ruta	Autorización	Integración
`GET /api/health`	NONE	`http://32.200.126.178:8080/api/health` (integración dedicada)
`ANY /{proxy+}`	JWT	`http://32.200.126.178:8080/{proxy}`
`OPTIONS /{proxy+}`	NONE	Mock / CORS

Dos bugs que hay que conocer:

1. **Sin `{proxy}` en la URI de integración**, API Gateway reenviaba `/api/pedidos` como `/` y el backend respondía 500 `"Error interno"` (el `GlobalExceptionHandler` se comía la excepción).
2. **El preflight `OPTIONS` caía en la ruta JWT** y devolvía 401. Se creó una ruta `OPTIONS /{proxy+}` con `--authorization-type NONE` , que ahora responde 200 con `access-control-allow-origin: *` .

5.3 Frontend

`frontend/src/authConfig.ts` incluye **metadata OIDC estática** porque el dominio de Cognito devolvía **404** en `/.well-known/openid-configuration` (problema de propagación de CloudFront):

```
metadata: ({
  issuer,
  authorization_endpoint: `${authority}/oauth2/authorize`,
  token_endpoint: `${authority}/oauth2/token`,
  userinfo_endpoint: `${authority}/oauth2/userInfo`,
  end_session_endpoint: `${authority}/logout`,
  jwks_uri: `${issuer}/.well-known/jwks.json`,
  ...
} as any)
```

Con eso el login funciona: botón → Hosted UI → credenciales → vuelta a la app con el JWT en memoria.

---

6. Evidencias de verificación (2026-11-02)

6.1 SSH a la instancia

```
$ ssh -i labsuser.pem ec2-user@32.200.126.178 'uname -a; docker --version; docker compose version'
Linux ip-172-31-26-83.ec2.internal 6.18.56-125.164.amzn2023.x86_64 ... x86_64 GNU/Linux
Docker version 25.0.14, build 0bab007
Docker Compose version v5.6.1
```

6.2 Contenedores

```
NAMES                    STATUS                   PORTS
pedidos360-frontend      Up (healthy)             0.0.0.0:5173->80/tcp
pedidos360-frontend-tls  Up                       0.0.0.0:8443->8443/tcp
pedidos360-backend       Up                       0.0.0.0:8080->8080/tcp
rabbitmq-2               Up                       0.0.0.0:5673->5672/tcp, 0.0.0.0:15673->15672/tcp
rabbitmq-1               Up (healthy)             0.0.0.0:5672->5672/tcp, 0.0.0.0:15672->15672/tcp
```

6.3 Clúster RabbitMQ

```
"cluster_name": "pedidos360-cluster",
"disk_nodes": ["rabbit@rabbitmq-1", "rabbit@rabbitmq-2"],
"running_nodes": ["rabbit@rabbitmq-2", "rabbit@rabbitmq-1"],
"partitions": {},
"versions": { "rabbit@rabbitmq-1": "3.13.7", "rabbit@rabbitmq-2": "3.13.7" }
```

6.4 Colas

```
name                   messages  consumers
documentos.dlq         0         0
tickets.dlq            1         0        <-- mensaje muerto de la demo
notificaciones.queue   0         1
tickets.queue          0         1
documentos.queue       0         1
notificaciones.dlq     0         0
```

6.5 API Gateway

```
GET  /prod/api/health   (sin token)  -> 200 {"status":"UP","app":"pedidos360-backend",...}
GET  /prod/api/pedidos  (sin token)  -> 401
GET  /prod/api/pedidos  (ADMIN)      -> 200 [ 5 pedidos ]
POST /prod/api/pedidos  (ADMIN)      -> 201 {"id":5,"producto":"Evidencia CLI",...}
GET  /prod/api/pedidos  (CUSTOMER)   -> 200
POST /prod/api/pedidos  (CUSTOMER)   -> 403
GET  /prod/api/admin/queues (CUSTOMER) -> 403
OPTIONS /prod/api/pedidos            -> 200 access-control-allow-origin: *
```

6.6 Frontend con login real

Navegando a `https://32.200.126.178:8443` e iniciando sesión como `admin@pedidos360.cl`:

```
PEDIDOS360  Panel de pedidos
admin@pedidos360.cl   [ADMIN]   Cerrar sesión
ACCESS TOKEN (JWT)  eyJraWQiOiJuNWd0YitvL0psNzA1NmF5RGVGdGFPWD…
API GATEWAY  https://bf0plxi897.execute-api.us-east-1.amazonaws.com/prod
1  Notebook   2  Demo            CREADO
2  Monitor    3  Cliente Gateway CREADO
3  Servidor 150  Cliente DLQ     CREADO
4  Teclado Mecanico 4 Cliente UI CREADO
```

Capturas: `evidencia_frontend.png` y `evidencia_crear.png`.

---

7. Cómo probarlo tú mismo (paso a paso)

Requisitos

- Las credenciales del lab **activas** (la instancia debe estar `running`).
- Navegador moderno.

7.1 Verificar que la instancia está arriba

```
aws ec2 describe-instances --region us-east-1 --instance-ids i-03a70f0a716b69333 \
  --query 'Reservations[0].Instances[0].[State.Name,PublicIpAddress]' --output text
# running   32.200.126.178
```

Si está `stopped`, arráncala:

```
aws ec2 start-instances --region us-east-1 --instance-ids i-03a70f0a716b69333
```

> Al detener y volver a arrancar, **la IP pública cambia**. En ese caso hay que actualizar la URI de integración del API Gateway y los callbacks de Cognito (ver §8).

7.2 Probar el backend directo

```
curl http://32.200.126.178:8080/api/health
# {"status":"UP","app":"pedidos360-backend",...}
```

7.3 Probar el API Gateway

```
curl https://bf0plxi897.execute-api.us-east-1.amazonaws.com/prod/api/health
curl -i https://bf0plxi897.execute-api.us-east-1.amazonaws.com/prod/api/pedidos   # 401
```

7.4 Obtener un token y llamar la API

```
TOKEN=$(aws cognito-idp initiate-auth \
  --client-id 261rvdvemjgib6n86vrb05tl9k \
  --auth-flow USER_PASSWORD_AUTH \
  --auth-parameters USERNAME=admin@pedidos360.cl,PASSWORD=Admin1234 \
  --query 'AuthenticationResult.IdToken' --output text)

curl -H "Authorization: Bearer $TOKEN" \
  https://bf0plxi897.execute-api.us-east-1.amazonaws.com/prod/api/pedidos
```

7.5 Probar el login en el navegador

1. Abre **https://32.200.126.178:8443**
2. Acepta la advertencia de certificado (*Avanzado → Continuar*).
3. Clic en **Iniciar sesión con Cognito**.
4. Usuario `admin@pedidos360.cl`, contraseña `Admin1234`.
5. Deberías volver a la app con el badge **ADMIN**, el JWT visible y la tabla de pedidos.
6. Crea un pedido y verifica que aparece en la tabla (POST 201 a través del gateway).

7.6 Probar los permisos por grupo

Repite el login con `cliente@pedidos360.cl` / `Cliente1234`: * Puede **ver** pedidos (GET 200). * El botón de crear devuelve **403**. * El endpoint `/api/admin/queues` devuelve **403**.

7.7 Probar RabbitMQ

- Consola nodo 1: `http://32.200.126.178:15672` → `pedidos` / `pedidos123`
- Consola nodo 2: `http://32.200.126.178:15673` → `pedidos` / `pedidos123`
- En *Queues and Streams* deberías ver las 6 colas y, en *Nodes*, los dos nodos del clúster.
- Para reproducir la DLQ: crea un pedido con `cantidad = 150` y observa cómo `tickets.dlq` sube a 1.

7.8 Probar la resiliencia del clúster

```
ssh -i labsuser.pem ec2-user@32.200.126.178
docker stop rabbitmq-2
docker exec rabbitmq-1 rabbitmqctl cluster_status   # queda 1 nodo corriendo
docker start rabbitmq-2
docker exec rabbitmq-1 rabbitmqctl cluster_status   # vuelven los 2
```

---

8. Operación y limpieza

8.1 Detener la instancia (importante para no agotar el lab)

```
aws ec2 stop-instances --region us-east-1 --instance-ids i-03a70f0a716b69333
```

O desde la consola: *EC2 → Instances →* `pedidos360-eval2` *→ Instance state → Stop instance*. Detenerla **no borra nada** (el disco EBS se conserva), solo deja de consumir el presupuesto de horas.

8.2 La IP es fija (Elastic IP)

La instancia tiene asociada la **Elastic IP** `32.200.126.178` (`eipalloc-04b1dfc182682f987`), por lo que **detener y volver a arrancar no cambia la IP** y no hay que reconfigurar nada.

> No liberes esa Elastic IP. Si se libera, el siguiente arranque asignará una IP nueva y habrá que repetir los pasos de §8.2-bis (API Gateway + Cognito + certificado + rebuild del frontend).

8.2-bis Si aun así cambia la IP

```
NEW_IP=$(aws ec2 describe-instances --region us-east-1 --instance-ids i-03a70f0a716b69333 \
  --query 'Reservations[0].Instances[0].PublicIpAddress' --output text)

# API Gateway
aws apigatewayv2 update-integration --api-id bf0plxi897 --integration-id <ID> \
  --integration-uri "http://$NEW_IP:8080/{proxy}"

# Cognito (callbacks)
aws cognito-idp update-user-pool-client --user-pool-id us-east-1_9b2C1fxtb \
  --client-id 261rvdvemjgib6n86vrb05tl9k \
  --callback-urls "[\"https://$NEW_IP:8443\",\"http://localhost:5173\"]" \
  --logout-urls  "[\"https://$NEW_IP:8443\",\"http://localhost:5173\"]" \
  --allowed-o-auth-flows code --allowed-o-auth-scopes openid email profile \
  --allowed-o-auth-flows-user-pool-client \
  --supported-identity-providers COGNITO \
  --explicit-auth-flows ALLOW_USER_PASSWORD_AUTH ALLOW_REFRESH_TOKEN_AUTH
```

Además hay que **regenerar el certificado autofirmado** con el nuevo `subjectAltName` y reconstruir el contenedor `frontend-tls`.

> **Nota**: ya está aplicado — la instancia usa Elastic IP, así que este procedimiento solo hace falta si alguien libera la EIP. En AWS Academy una EIP asociada a una instancia corriendo no tiene costo.

8.3 Borrar todo

```
aws cloudformation delete-stack --region us-east-1 --stack-name pedidos360-eval2
aws cognito-idp delete-user-pool --user-pool-id us-east-1_9b2C1fxtb
aws apigatewayv2 delete-api --api-id bf0plxi897
```

8.4 Seguridad

- El **PAT de GitHub** que se usó para el despliegue debe **revocarse** en *GitHub → Settings → Developer settings → Personal access tokens*.
- Las credenciales de AWS Academy expiran solas; no quedan guardadas en el repo.
- El certificado TLS es autofirmado: sirve para la demo, no para producción.

---

9. Diferencias entre lo documentado antes y la realidad

Punto	Guía original	Realidad desplegada
Frontend	`http://<ip>:5173`	**`https://<ip>:8443`** (Cognito exige HTTPS)
Callback Cognito	`http://...:5173`	`https://32.200.126.178:8443`
App Client	creado por `02-cognito.sh`	`261rvdvemjgib6n86vrb05tl9k` (`pedidos360-spa-v2`)
Integración API GW	`http://<ip>:8080`	`http://32.200.126.178:8080/{proxy}`
Ruta OPTIONS	no existía	`OPTIONS /{proxy+}` sin autorizador
Metadata OIDC	descubrimiento automático	metadata estática en `authConfig.ts`
Memoria backend	sin límite	`-Xmx256m`, `mem_limit: 460m` + 2 GB swap
Usuario de prueba CUSTOMER	`customer@`	**`cliente@pedidos360.cl`**